МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №3
по курсу «Программирование графических процессоров»

Классификация и кластеризация изображений на GPU.

Выполнил: Немкова А. Р.
Группа: М8О-408Б-22
Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2025

Условие

Цель работы. Научиться использовать GPU для классификации и кластеризации изображений. Использование константной памяти и одномерной сетки потоков.

Вариант №2. Метод расстояния Махаланобиса.

Программное и аппаратное обеспечение

Графический процессор:

- Модель: NVIDIA GeForce RTX 3050 Laptop GPU
- Compute Capability: 8.6
- Объем графической памяти: 4 294 443 008
- Разделяемая память на блок: 49 152
- Количество регистров на блок: 65 536
- Размер warp: 32
- Максимальное количество потоков на блок: (1024 × 1024 × 64)
- Максимальное количество блоков: (2 147 483 647 × 65 535 × 65 535)
- Объем константной памяти: 65 536
- Количество мультипроцессоров (SM): 16

Центральный процессор:

- Модель: AMD Ryzen 7 5800H with Radeon Graphics 3.20 GHz
- Количество ядер: 8
- Количество потоков: 16
- Базовая тактовая частота: 3,2 ГГц
- Максимальная тактовая частота: 4,4 ГГц
- Кеш L1: 64 Кб (на ядро)
- Кеш-память L2: 512 КБ (на ядро)
- Кеш L3: 16 Мб

Оперативная память:

- Тип памяти: DDR4
- Объём памяти: 2 * 8 Гб
- Частота: 3200 Мт/с

Жёсткий диск:

- Тип: SSD
- Модель: SK Hynix HFM001TD3JX013N
- Объём памяти: 1 Тб
- Скорость записи/чтения: 3100/3500 МБ/с

Программное обеспечение:

- OS: Ubuntu 20.04 Linux (WSL2)
- IDE: VS Code
- Compiler: nvcc, gcc

Метод решения

Алгоритм работает в два этапа: на этапе инициализации происходит вычисление статистических характеристик классов по предоставленным выборкам пикселей, а на этапе выполнения осуществляется параллельная классификация всех пикселей изображения.

Для каждого класса вычисляется вектор средних значений:

$$avg_j = \frac{1}{np_j}\sum_{i=1}^{np_j} ps_i^j$$

где $ps_i^j = (r_i^j, g_i^j, b_i^j)^T -$ i-ый пиксель j-ой выборки, а $np_j -$ количество пикселей в выборке. Затем рассчитывается ковариационная матрица:

$$cov_j = \frac{1}{np_j - 1}\sum_{i=1}^{np_j}(ps_i^j - avg_j)(ps_i^j - avg_j)^T$$

После этого через определитель вычисляется обратная ковариационная матрица $cov_j{}^{-1}$.

Все вычисленные параметры копируются в константную память GPU. Затем запускается ядро, где для каждого пикселя p = (r, g, b) вычисляется расстояние Махаланобиса до каждого класса:

$$d_j = (\mathrm{p} - avg_j)^T * cov_j{}^{-1} * (\mathrm{p} - avg_j)$$

Классификация выполняется по правилу:

$$jc = \underset{J}{\mathrm{argmax}}[-d_j]$$

При равенстве расстояний выбирается класс с наименьшим номером. Номер выбранного класса записывается в альфа-канал соответствующего пикселя выходного изображения.

Описание программы

Программа реализована в виде одного файла, содержащего 7 функций и 2 макроса.

Основные типы данных:

- uchar4 - 4-компонентный вектор беззнаковых символов для хранения пикселей формата RGBA
- float* - указатели на массивы для хранения статистических параметров классов
- float[3] - векторы для RGB-компонент и разностей
- float[9] - матрицы 3×3 для ковариационных матриц и обратных к ним

Основные функции:

- Макрос CSC - автоматическая проверка ошибок CUDA с выводом диагностической информации
- kernel - ядро для параллельной классификации пикселей: принимает указатель на данные изображения, размеры и количество классов; реализует grid-stride loops для обработки всех пикселей изображения; для каждого пикселя вычисляет расстояния Махаланобиса до всех классов; выбирает класс с минимальным расстоянием и записывает его номер в альфа-канал
- det_matrix - вычисление определителя матрицы 3×3
- inverse_matrix - вычисление обратной матрицы
- avg_vector - вычисление вектора средних значений
- cov_matrix - вычисление ковариационной матрицы

- main - основная логика программы: чтение входных данных и изображения; вычисление статистик для каждого класса; копирование параметров в константную память GPU; запуск ядра и сохранение результата

Константная память GPU:

- const_avg - массив векторов средних значений всех классов
- const_cov_inv - массив обратных ковариационных матриц всех классов

Результаты

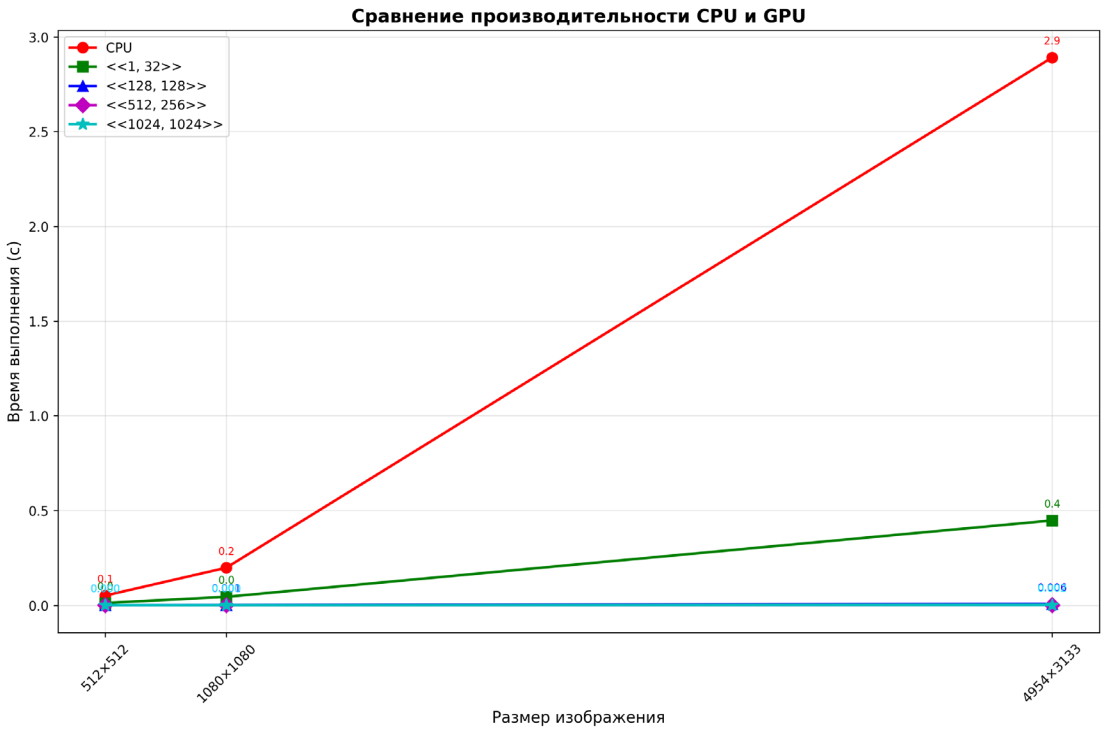
Были проведены тесты производительности GPU ядра с различными конфигурациями запуска для изображений разных размеров.

Конфигурация	512 x 512	1080 x 1080	4954 x 3133
<<1, 32 >>	0.011761 s	0.044159 s	0.447278 s
<<32, 64 >>	0.000347 s	0.000822 s	0.006271 s
<<128, 128 >>	0.000229 s	0.000335 s	0.002070 s
<<512, 256 >>	0.000237 s	0.000326 s	0.001811 s
<<1024, 1024 >>	0.000256 s	0.000363 s	0.001868 s

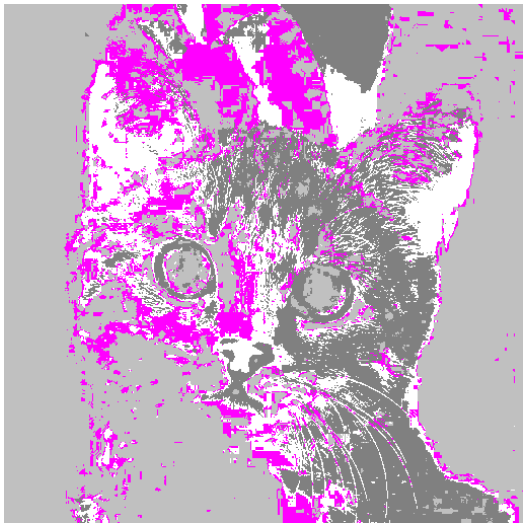
Также были выполнены замеры работы с использованием CPU

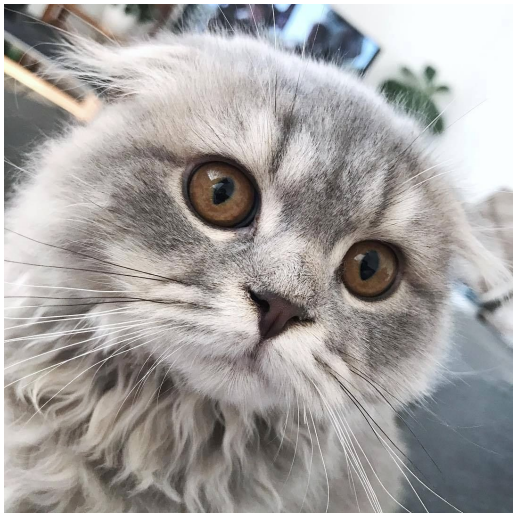
Размер изображения	Время работы
512 x 512	0.050796 s
1080 x 1080	0.197828 s
4954 x 3133	2.890707 s

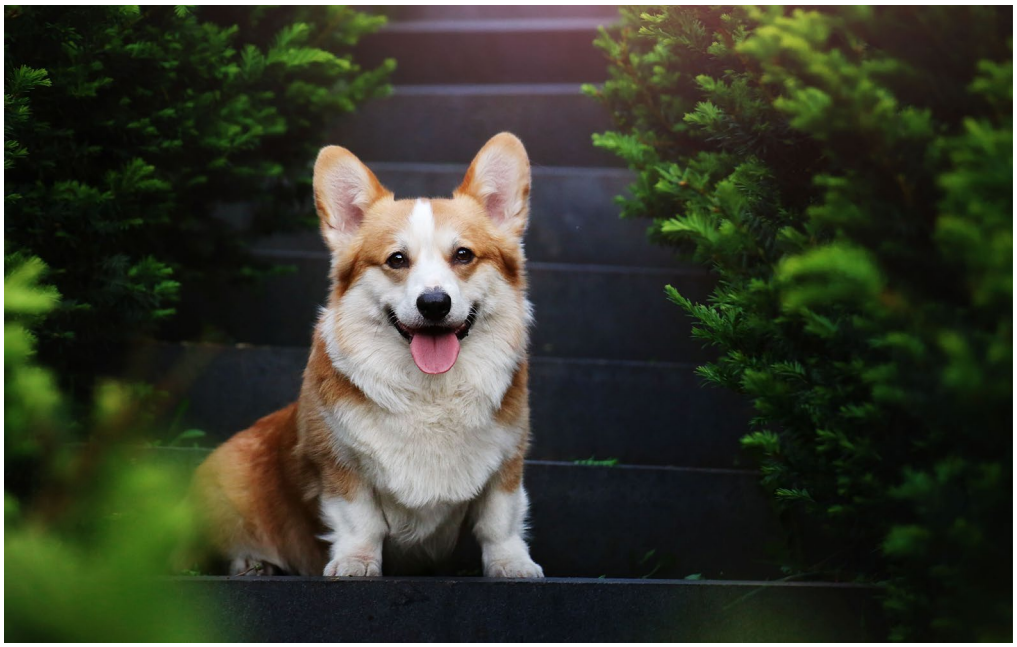
Ниже представлен график сравнения замеров времени работы GPU с конфигурациями <<1, 32 >>, <<128, 128 >>, <<512, 256 >>, <<1024, 1024 >> и CPU на тестах с размерами изображений 512 x 512, 1080 x 1080 и 4954 x 3133.



Примеры получившихся изображений







Выводы

В ходе данной лабораторной работы был реализован алгоритм классификации пикселей изображения методом расстояния Махаланобиса с использованием технологии CUDA.

Сравнение производительности показало, что оптимальной конфигурацией оказалась <<<512, 256>>>, которая обеспечила наилучшее время выполнения для большинства тестовых изображений. Более высокие конфигурации, такие как <<<1024, 1024>>>, показали результаты сопоставимые или незначительно лучше, но при этом потребляют значительно больше ресурсов GPU. Меньшие конфигурации, например <<<1, 32>>> и <<<32, 64>>>, демонстрируют существенно более низкую производительность из-за недостаточного использования параллельных возможностей графического ускорителя.

Данный алгоритм находит применение в задачах компьютерного зрения и машинного обучения, где требуется сегментация изображений на основе статистических характеристик, анализ текстур, медицинская диагностика по снимкам, а также в системах автоматического контроля качества в промышленности. Метод Махаланобиса особенно эффективен в случаях, когда классы имеют различные ковариационные структуры и требуют учета корреляций между цветовыми каналами для точной классификации.